DATABASE MANAGEMENT SYSTEMS

26.03.2025 **LAB MANUAL** **23BAI10917**

VIJAY RAJESH R

1. Consider the following relations containing airline flight information:

Flights(flno: integer, from: string, to: string,distance: integer, departs: time, arrives: time)

Aircraft(aid: integer, aname: string, cruisingrange: integer)

Certified(eid: integer, aid: integer)

Employees(eid: integer, ename: string, salary: integer)

Note that the Employees relation describes pilots and other kinds of employees as well every pilot is certified for some aircraft (otherwise, he or she would not qualify as a pilot), and only pilots are certified to fly.

Write the following queries in SQL

1. Find the eids of pilots certified for some Boeing aircraft.
2. Find the names of pilots certified for some Boeing aircraft.
3. Find the aids of all aircraft that can be used on non-stop flights from Bonn to Madras.
4. Identify the flights that can be piloted by every pilot whose salary is more than $100,000.
5. Find the names of pilots who can operate planes with a range greater than 3,000 miles but are not certified on any Boeing aircraft.

Solution:

```
1. SELECT DISTINCT C.eid
   FROM Certified C
   JOIN Aircraft A ON C.aid = A.aid
   WHERE A.aname LIKE 'Boeing%';
2. SELECT DISTINCT E.ename
   FROM Employees E
   JOIN Certified C ON E.eid = C.eid
```

```
    JOIN Aircraft A ON C.aid = A.aid
    WHERE A.aname LIKE 'Boeing%';
3. SELECT DISTINCT A.aid
    FROM Aircraft A
    JOIN Flights F ON F.distance <= A.cruisingrange
    WHERE F.from = 'Bonn' AND F.to = 'Madras';
4. SELECT F.flno FROM Flights F
    WHERE NOT EXISTS (
      SELECT E.eid
      FROM Employees E
      WHERE E.salary > 100000
      AND NOT EXISTS (
        SELECT C.aid
        FROM Certified C
        JOIN Aircraft A ON C.aid = A.aid
        WHERE C.eid = E.eid
        AND A.aid IN (SELECT A2.aid FROM Aircraft A2 WHERE A2.aid IN
    (SELECT F2.aid FROM Flights F2 WHERE F2.flno = F.flno))
      )
    );
5. SELECT DISTINCT E.ename FROM
    Employees E
    JOIN Certified C ON E.eid = C.eid
    JOIN Aircraft A ON C.aid = A.aid
    WHERE A.cruisingrange > 3000
    AND E.eid NOT IN (
      SELECT C2.eid
      FROM Certified C2
      JOIN Aircraft A2 ON C2.aid = A2.aid
      WHERE A2.aname LIKE 'Boeing%'
    );
```

2. SAILORS (SID:INTEGER, SNAME:STRING, RATING:INTEGER, AGE:REAL)

BOATS (BID:INTEGER, BNAME:STRING, COLOR:STRING)

RESERVES (SID:INTEGER, BID:INTEGER, DAY:DATE)

1. Display names & ages of all sailors.
2. Find all sailors with a rating above 7.
3. Display all the names & colors of the boats.
4. Find all the boats with Red color.
5. Find the names of sailors who have reserved boat number

123.

6. Find SIDs of sailors who have reserved Pink Boat;
7. Find the color of the boats reserved by Rajesh.

8. Find names of the sailors who have reserved at least one boat.
9. Find the names of sailors who have reserved a red or a green boat.
10. Find the names of sailors who have reserved boat 103.
11. Find the names of sailors who have not reserved boat 103.
12. Find sailors whose rating is better than some sailor called Rajesh.
13. Find the sailor's with the highest rating using ALL.
14. To count number SIDs of sailors in Sailors table 15. To count numbers of boats booked in Reserves table.
16. To count number of Boats in Boats table.
17. To find age of Oldest Sailor.
18. To find age of Youngest Sailor.
19. Find the average age of sailors with a rating of 10.
20. Count the number of different sailor names.
21. Find the name and age of the oldest sailor.
22. Count the number of Sailors.
23. Find the names of sailors who are older than the oldest sailor with a rating of 10.
24. Display all the sailors according to their ages.
25. To display names of sailors according to alphabetical order.

Solution:

1. SELECT SNAME, AGE FROM SAILORS;
2. SELECT * FROM SAILORS WHERE RATING > 7;
3. SELECT BNAME, COLOR FROM BOATS;
4. SELECT * FROM BOATS WHERE COLOR = 'Red';
5. SELECT DISTINCT S.SNAME FROM SAILORS S JOIN RESERVES R ON S.SID = R.SID WHERE R.BID = 123;
6. SELECT DISTINCT R.SID FROM RESERVES R JOIN BOATS B ON R.BID = B.BID WHERE B.COLOR = 'Pink';
7. SELECT DISTINCT B.COLOR FROM BOATS B JOIN RESERVES R ON B.BID = R.BID JOIN SAILORS S ON R.SID = S.SID WHERE S.SNAME = 'Rajesh';
8. SELECT DISTINCT S.SNAME FROM SAILORS S JOIN RESERVES R ON S.SID = R.SID;

9. SELECT DISTINCT S.SNAME FROM SAILORS S JOIN RESERVES R ON S.SID = R.SID JOIN BOATS B ON R.BID = B.BID WHERE B.COLOR IN ('Red', 'Green');
10. SELECT DISTINCT S.SNAME FROM SAILORS S JOIN RESERVES R ON S.SID = R.SID WHERE R.BID = 103;
11. SELECT SNAME FROM SAILORS WHERE SID NOT IN (SELECT SID FROM RESERVES WHERE BID = 103);
12. SELECT DISTINCT S1.SNAME FROM SAILORS S1 WHERE S1.RATING > (SELECT MIN(S2.RATING) FROM SAILORS S2 WHERE S2.SNAME = 'Rajesh');
13. SELECT SNAME FROM SAILORS WHERE RATING >= ALL (SELECT RATING FROM SAILORS);
14. SELECT COUNT(SID) FROM SAILORS;
15. SELECT COUNT(DISTINCT BID) FROM RESERVES;
16. SELECT COUNT(BID) FROM BOATS;
17. SELECT MAX(AGE) FROM SAILORS;
18. SELECT MIN(AGE) FROM SAILORS;
19. SELECT AVG(AGE) FROM SAILORS WHERE RATING = 10;
20. SELECT COUNT(DISTINCT SNAME) FROM SAILORS;
21. SELECT SNAME, AGE FROM SAILORS WHERE AGE = (SELECT MAX(AGE) FROM SAILORS);
22. SELECT COUNT(*) FROM SAILORS;
23. SELECT SNAME FROM SAILORS WHERE AGE > (SELECT MAX(AGE) FROM SAILORS WHERE RATING = 10);
24. SELECT * FROM SAILORS ORDER BY AGE;
25. SELECT SNAME FROM SAILORS ORDER BY SNAME;

3. Design the data base for a wholesale furniture company. The database has to allow to analyze the company's situation at least with respect to the Furniture, Customers and Time. Moreover, the company needs to analyze:
the furniture with respect to its type (chair, table, wardrobe, cabinet. . .), category (kitchen, living room, bedroom, bathroom, office. . .) and material (wood, marble. . .) the customers with respect to their spatial location, by considering at least cities, regions and states The company is

interested in learning at least the quantity, income and discount of its sales.

Solution:

```
CREATE TABLE Furniture (
fid INT PRIMARY KEY,
fname VARCHAR(255),    type
VARCHAR(50),        category
VARCHAR(50),     material
VARCHAR(50)
);

CREATE TABLE Customers (
cid INT PRIMARY KEY,
cname VARCHAR(255),
city VARCHAR(100),    region
VARCHAR(100),    state
VARCHAR(100)
);

CREATE TABLE Sales (
sale_id INT PRIMARY KEY,
   fid INT,    cid INT,
sale_date DATE,    quantity
INT,    income
DECIMAL(10,2),
discount DECIMAL(5,2),
   FOREIGN KEY (fid) REFERENCES Furniture(fid),
   FOREIGN KEY (cid) REFERENCES Customers(cid)
);
```

```
CREATE TABLE Time (
time_id INT PRIMARY KEY,
sale_date DATE UNIQUE,
   year INT,
quarter INT,
month INT,     day
INT,
week_of_year INT
);
```

4. Simple script to backup all SQL server database

Create a database table with the following fields:

Field name Data type

Ship_id Number -- This is the ID of a particular Ship

Date_expected Date --The date at which the goods are expected to

arrive

Qty_expected Number --The quantity that is supposed to arrive

Description Varchar2 --The description of the items

Color Varchar2 --The color of the items

Qty_hand Number –The quantity on hand for these items

Itemrate Number—Price of each item.

Write a PL/SQL program that uses implicit cursor to display the data expected, quantity expected, item description, color and quantity on hand for any particular Ship ID number.

Solution:

```
-- Step 1: Create the Shipment Table
CREATE TABLE Shipments (
   Ship_id NUMBER PRIMARY KEY,
   Date_expected DATE,
   Qty_expected NUMBER,
   Description VARCHAR2(255),
```

```
    Color VARCHAR2(50),
    Qty_hand NUMBER,
    Itemrate NUMBER
);

-- Step 2: PL/SQL Program using Implicit Cursor
DECLARE
    v_Date_expected Shipments.Date_expected%TYPE;     v_Qty_expected
Shipments.Qty_expected%TYPE;     v_Description
Shipments.Description%TYPE;     v_Color Shipments.Color%TYPE;
v_Qty_hand Shipments.Qty_hand%TYPE;     v_Ship_id NUMBER :=
&Enter_Ship_ID;  -- Accept user input for Ship ID

BEGIN
    -- Implicit Cursor: Select and Display Shipment Data
    FOR shipment_rec IN (SELECT Date_expected, Qty_expected, Description, Color,
Qty_hand
                         FROM Shipments
                         WHERE Ship_id = v_Ship_id) LOOP
        DBMS_OUTPUT.PUT_LINE('Date Expected: ' || shipment_rec.Date_expected);
        DBMS_OUTPUT.PUT_LINE('Quantity Expected: ' || shipment_rec.Qty_expected);
        DBMS_OUTPUT.PUT_LINE('Description: ' || shipment_rec.Description);
        DBMS_OUTPUT.PUT_LINE('Color: ' || shipment_rec.Color);
        DBMS_OUTPUT.PUT_LINE('Quantity on Hand: ' || shipment_rec.Qty_hand);
    END LOOP;
END;
/

DECLARE @BackupPath NVARCHAR(255) = 'C:\SQLBackups\';
DECLARE @DatabaseName NVARCHAR(255);
DECLARE @BackupCommand NVARCHAR(500);
```

```
DECLARE db_cursor CURSOR FOR
SELECT name FROM sys.databases WHERE database_id > 4;  -- Exclude system databases

OPEN db_cursor;
FETCH NEXT FROM db_cursor INTO @DatabaseName;

WHILE @@FETCH_STATUS = 0
BEGIN
   SET @BackupCommand = 'BACKUP DATABASE [' + @DatabaseName + '] TO DISK = '''
+ @BackupPath + @DatabaseName + '_Backup.bak''';
   EXEC sp_executesql @BackupCommand;

   FETCH NEXT FROM db_cursor INTO @DatabaseName;
END

CLOSE db_cursor;
DEALLOCATE db_cursor;
```

5. Create a transparent audit system for a table Client_master (client_no, name, address, Bal_due). The system must keep track of the records that are being deleted or updated. The functionality being when a record is deleted or modified, the original record details and the date of operation are stored in the auditclient(client_no, name, bal_due, operation, userid, update) table, then the delete or update is allowed to go through.

Solution:

```
CREATE TABLE auditclient (
client_no   VARCHAR(20),    name
VARCHAR(100),    bal_due
DECIMAL(10,2), operation
VARCHAR(10),    userid
VARCHAR(50),
   update_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);
CREATE OR REPLACE TRIGGER trg_client_audit
BEFORE UPDATE OR DELETE ON Client_master
FOR EACH ROW DECLARE
   v_user VARCHAR(50);
BEGIN
   -- Get the current user
   SELECT USER INTO v_user FROM DUAL;

   -- Insert old data into the audit table
   INSERT INTO auditclient (client_no, name, bal_due, operation, userid, update_time)
   VALUES (:OLD.client_no, :OLD.name, :OLD.bal_due, CASE WHEN DELETING THEN
'DELETE' ELSE 'UPDATE' END, v_user, SYSDATE);
END;
```

6. Using the supplier and parts database, write an cursor program to read and print all parts in part number, deleting every tenth one as you go, and begin a new transaction after every tenth row. You can use the foreign key delete CASCADE rule from parts,commit,rollback and savepoint.

Solution:

```
DECLARE
   CURSOR part_cursor IS SELECT part_no FROM Parts;
v_part_no Parts.part_no%TYPE;     v_count NUMBER :=
0;
BEGIN
   FOR part_rec IN part_cursor LOOP
v_count := v_count + 1;

      IF MOD(v_count, 10) = 0 THEN
         DELETE FROM Parts WHERE part_no = part_rec.part_no;
         COMMIT;  -- Commit after every 10th row
      END IF;
   END LOOP;
```

```
    DBMS_OUTPUT.PUT_LINE('Every 10th part deleted successfully.');
END;
/
```

7 Assuming a patient should not receive both treatment and prescription from the same doctor, write a program to find out all the doctor who provide both treatment and prescription to the same patient. In addition, raise and display an exception if this situation occurs.

Solution:

```
DECLARE
   CURSOR doctor_cursor IS
      SELECT t.doctor_id, t.patient_id
      FROM treatment t
      JOIN prescription p ON t.patient_id = p.patient_id AND t.doctor_id = p.doctor_id;

   v_doctor_id NUMBER;
v_patient_id NUMBER;

   ex_duplicate EXCEPTION;
BEGIN
   FOR rec IN doctor_cursor LOOP
v_doctor_id := rec.doctor_id;        v_patient_id
:= rec.patient_id;

      RAISE_APPLICATION_ERROR(-20001, 'Doctor ' || v_doctor_id || ' provides both
treatment and prescription to patient ' || v_patient_id);
   END LOOP;
END;
/
```

8. Write a PL/SQL block which includes a procedure getCleanerDetails which accepts a cleaner number and returns the cleaners name and salary. Create a stored function called getCleanersLocation. This function takes as input a cleaner's number and returns the cleaner's depot address. Call the function from within an SQL statement to select the cleaner's name and location for a particular cleaner.

Solution:

```
CREATE OR REPLACE PROCEDURE getCleanerDetails(
p_cleaner_no IN NUMBER,     p_name OUT VARCHAR2,
p_salary OUT NUMBER
) AS
BEGIN
   SELECT name, salary INTO p_name, p_salary FROM Cleaners WHERE cleaner_no =
p_cleaner_no;
END;
/
CREATE OR REPLACE FUNCTION getCleanersLocation(p_cleaner_no NUMBER)
RETURN VARCHAR2 AS
   v_location VARCHAR2(100);
BEGIN
   SELECT depot_address INTO v_location FROM Cleaners WHERE cleaner_no =
p_cleaner_no;
   RETURN v_location;
END;
/
SELECT name, getCleanersLocation(113) FROM Cleaners WHERE cleaner_no = 113;
```

9 Write a PL/SQL block which includes a procedure getCleanerDetails which accepts a cleaner number and returns the cleaners name and salary. The main block should call the procedure with cleaner number '113' and output this cleaner's details including the salary which has been increased by 10%.

Solution:

```
DECLARE
   v_name VARCHAR2(50);
v_salary NUMBER; BEGIN
   getCleanerDetails(113, v_name, v_salary);
v_salary := v_salary * 1.10; -- Increase by 10%

   DBMS_OUTPUT.PUT_LINE('Cleaner: ' || v_name || ', New Salary: ' || v_salary);
END;
/
```

10 Create a Trigger that raises an User Defined Error Message and does not allow the update and Insert operation in the database.

Solution:

```
CREATE OR REPLACE TRIGGER prevent_modifications
BEFORE INSERT OR UPDATE ON Employees
FOR EACH ROW
BEGIN
   RAISE_APPLICATION_ERROR(-20002, 'Insert and Update operations are not allowed.');
END;
/
```

11. Join Queries: Assume necessary database schema

Display the name of each employee with his department name.

Display a list of all departments with the employees in each department.

Display all the departments with the manager for that department.

Display the names of each employee with the name of his/her boss.

Display the names of each employee with the name of his/her boss with a blank for the boss of the president.

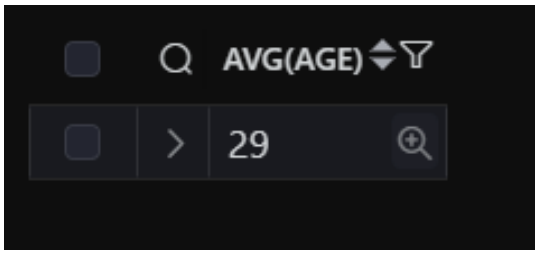
Display the employee number and name of each employee who manages other employees with the number of people he or she manages.

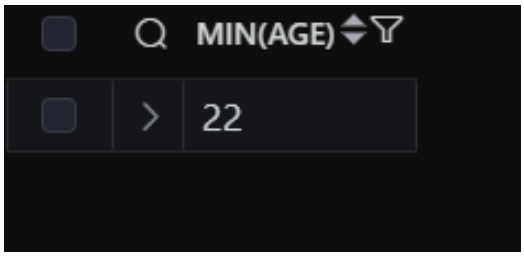
Repeat the display for the last question, but this time display the rows in descending order of the number of employees managed.

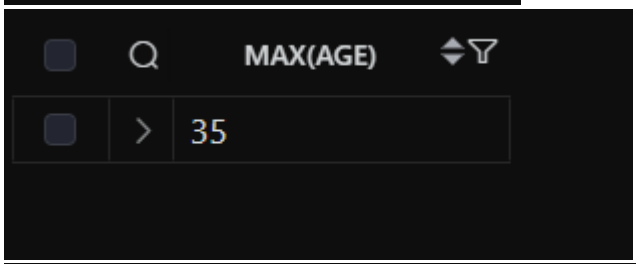
```
-- 1. Employee name with department name
SELECT e.name AS employee_name, d.department_name AS department_name, NULL AS manager_name, NULL AS boss_name, NULL AS num_employees
FROM Employees e
JOIN Departments d ON e.department_id = d.department_id

UNION ALL

-- 2. All departments with employees
SELECT e.name AS employee_name, d.department_name AS department_name, NULL AS manager_name, NULL AS boss_name, NULL AS num_employees
FROM Departments d
LEFT JOIN Employees e ON d.department_id = e.department_id

UNION ALL

-- 3. Departments with managers
SELECT NULL AS employee_name, d.department_name AS department_name, m.name AS manager_name, NULL AS boss_name, NULL AS num_employees
FROM Departments d
JOIN Employees m ON d.manager_id = m.employee_id

UNION ALL

-- 4. Employee name with boss name
SELECT e.name AS employee_name, NULL AS department_name, NULL AS manager_name, m.name AS boss_name, NULL AS num_employees
FROM Employees e
LEFT JOIN Employees m ON e.manager_id = m.employee_id

UNION ALL

-- 5. Employees with blank for president's boss
```

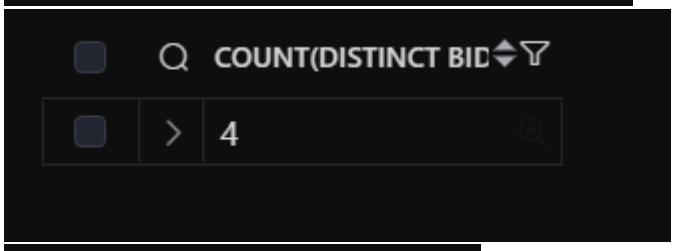
```
SELECT e.name AS employee_name, NULL AS department_name, NULL AS manager_name,
COALESCE(m.name, ' ') AS boss_name, NULL AS num_employees

FROM Employees e

LEFT JOIN Employees m ON e.manager_id = m.employee_id


UNION ALL


-- 6. Employee number, name, and number of employees managed

SELECT e.name AS employee_name, NULL AS department_name, NULL AS manager_name,
NULL AS boss_name, COUNT(emp.employee_id) AS num_employees

FROM Employees e

JOIN Employees emp ON e.employee_id = emp.manager_id

GROUP BY e.employee_id, e.name


UNION ALL


-- 7. Same as above, sorted by employees managed in descending order

SELECT e.name AS employee_name, NULL AS department_name, NULL AS manager_name,
NULL AS boss_name, COUNT(emp.employee_id) AS num_employees

FROM Employees e

JOIN Employees emp ON e.employee_id = emp.manager_id

GROUP BY e.employee_id, e.name

ORDER BY num_employees DESC;
```

Screenshots/Output:

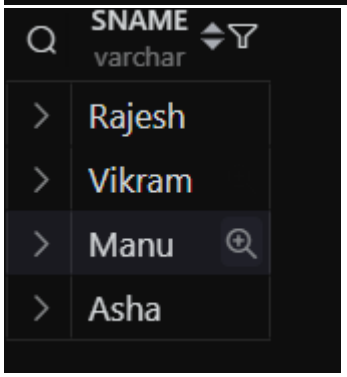


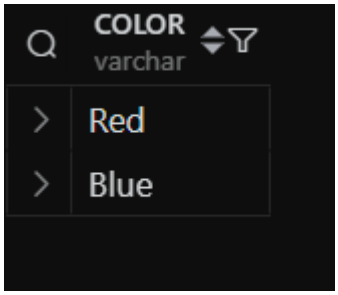

MIN(AGE)
22

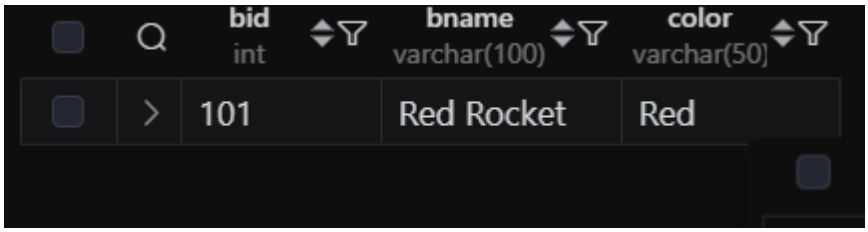

MAX(AGE)
35


COUNT(DISTINCT BID
4


COUNT(SID)
4


SNAME
varchar
Rajesh
Vikram
Manu
Asha

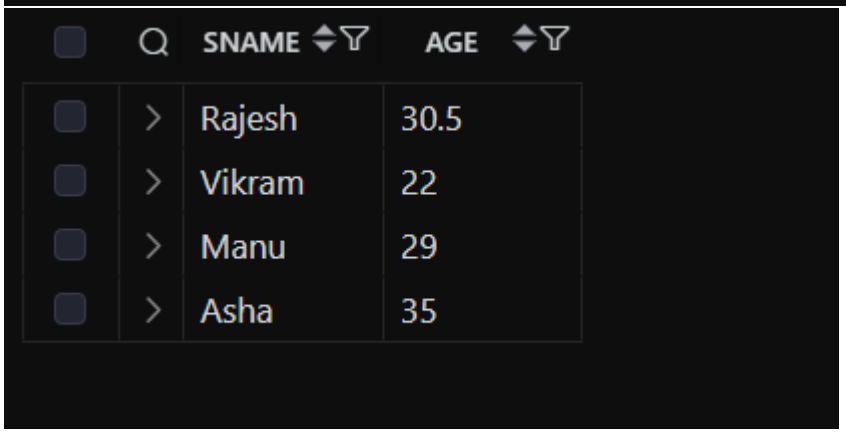

COLOR
varchar
Red
Blue



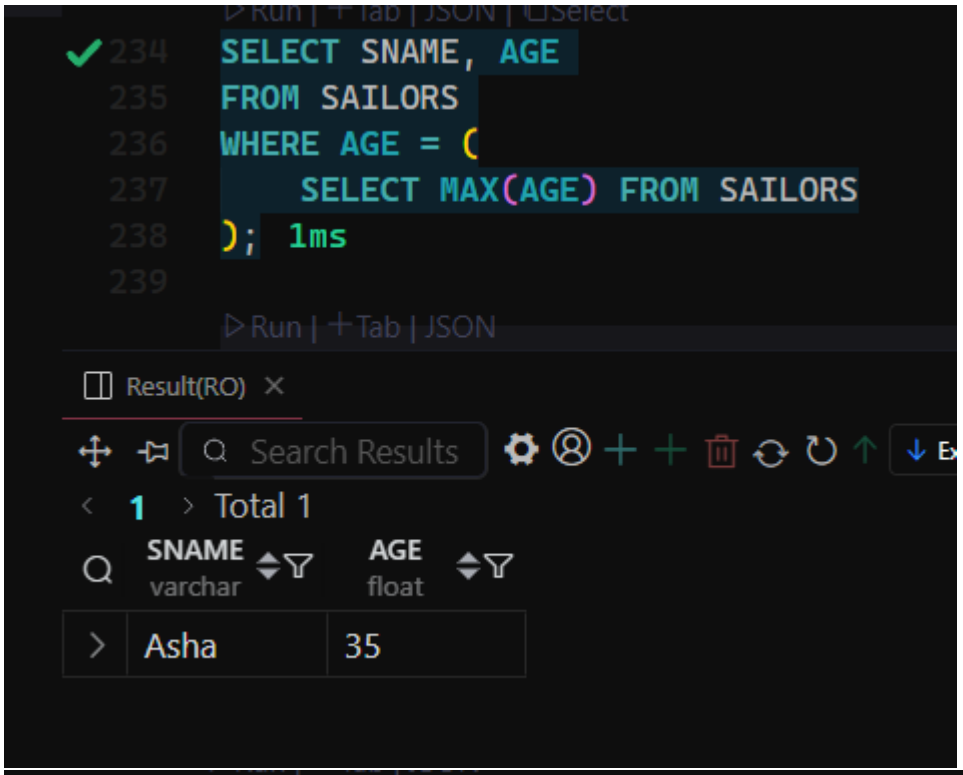
bid int	bname varchar(100)	color varchar(50)
101	Red Rocket	Red

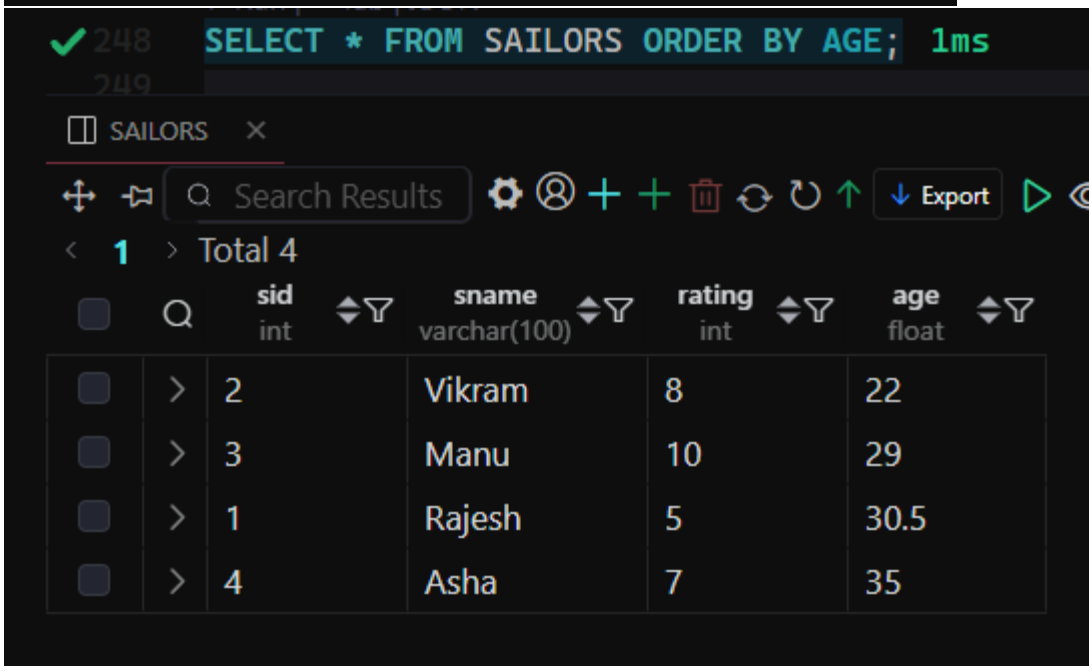
BNAME	COLOR
Red Rocket	Red
Blue Moon	Blue
Green Dragc	Green
Pink Panther	Pink

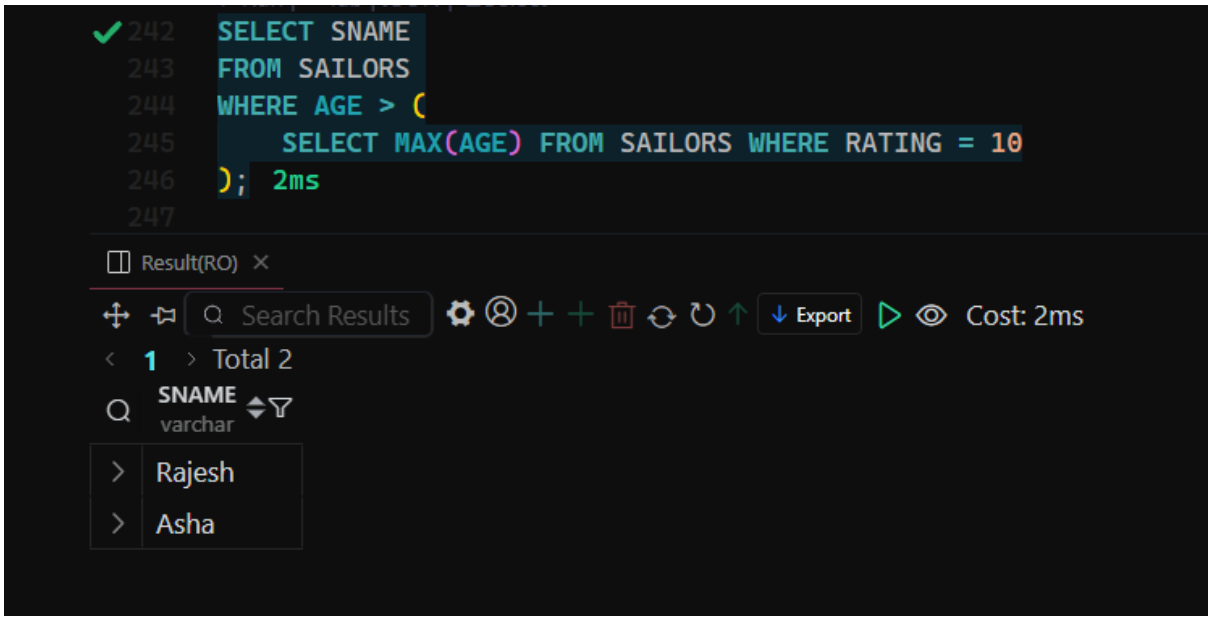
sid int	sname varchar(100)	rating int	age float
2	Vikram	8	22
3	Manu	10	29



SNAME	AGE
Rajesh	30.5
Vikram	22
Manu	29
Asha	35


SELECT SNAME, AGE
FROM SAILORS
WHERE AGE = (
SELECT MAX(AGE) FROM SAILORS
); 1ms
Run | Tab | JSON
Result(RO)
Search Results
1
Total 1
SNAME
varchar
AGE
float
Asha
35


SELECT * FROM SAILORS ORDER BY AGE; 1ms
SAILORS
Search Results
Export
1
Total 4
sid
int
sname
varchar(100)
rating
int
age
float
2
Vikram
8
22
3
Manu
10
29
1
Rajesh
5
30.5
4
Asha
7
35


242 SELECT SNAME
243 FROM SAILORS
244 WHERE AGE > (
245 SELECT MAX(AGE) FROM SAILORS WHERE RATING = 10
246); 2ms
247
Result(RO)
Search Results
Export
Cost: 2ms
1
Total 2
SNAME
varchar
Rajesh
Asha